University of Florence
Engineering School of Florence
Department of Information Engineering

Sviluppo di un Modulo di Interazione Vocale per un Sistema Conversazionale Multi-Agente Orientato all'Automazione

Author:
Niccolò Benedetto

Supervised by:
Prof. Pietro Pala
Prof. Andrea Ferracani

Academic Year: 2024-2025

Dedication

A voi, mia famiglia, che mai avete vacillato nella fede che riponevate in me, che con sguardi silenziosi e sorrisi discreti mi avete sempre sorretto: il vostro supporto è stato, e sarà, la mia roccia taciturna.

Ma oggi, in questa pagina che segna la fine di un lungo capitolo, rivolgo il mio più sentito ringraziamento a me stesso.
A me, che ho scelto di intraprendere questo cammino impervio, accettando la fatica come unica alleata.
A me, che tra svariati fallimenti e qualche incertezza, mai ho smesso di credere nella forza della mia testa, nella fermezza di ogni mio passo e nel valore di ciascun sacrificio.
A me, che ho costruito pietra dopo pietra, con pazienza e ostinazione, ciò che oggi posso chiamare conquista.

Questa tesi non è soltanto la prova tangibile del sapere acquisito, tuttavia è anche il simbolo di un cammino compiuto con dignità e coraggio. È il punto culminante di una fase e, al contempo, l'alba di una nuova partenza, più ambiziosa e consapevole. Che ogni fatica fin qui sostenuta sia il fondamento su cui poggiare il futuro, e che ogni sogno diventi progetto, ogni progetto, realtà.

A chi crede nel valore del lavoro silenzioso, dell'impegno incrollabile e della volontà che non chiede aiuto ma cerca solo se stessa: questa pagina è per voi.
Ma, prima di tutti, è per me.

Nonno, ce l'ho fatta. So che hai visto tutto, e che oggi sorridi con me.

Acknowledgments

Vorrei esprimere la mia sincera gratitudine ai professori Pietro Pala e Andrea Ferracani, la cui tempestiva guida e competenza sono state determinanti nel corso di questa tesi.

Ringrazio di cuore la mia famiglia per il costante incoraggiamento e la comprensione. Infine, cito il laboratorio MICC per aver messo a disposizione il framework Genhapi su cui ho avuto il piacere di lavorare.

Abstract

Negli ultimi anni i sistemi conversazionali hanno assunto un ruolo centrale nei processi di automazione e supporto decisionale, grazie all'impiego di architetture multi-agente e modelli linguistici di grandi dimensioni.

In questo lavoro viene presentato lo sviluppo e l'integrazione di un modulo di interazione vocale per Genhapi, un sistema conversazionale multi-agente orientato all'automazione, sviluppato presso il laboratorio MICC dell'Università di Firenze. Il modulo consente all'utente di dialogare in linguaggio naturale attraverso input e output vocali, mantenendo la compatibilità con i flussi di orchestrazione e automazione già presenti nel sistema.

La soluzione proposta si basa su componenti e modelli open-source per la trascrizione del parlato (speech-to-text) e la sintesi vocale (text-to-speech), integrati in un flusso che permette di trasformare l'audio in comandi strutturati e di restituire risposte vocali generate dal sistema. L'implementazione è stata progettata per garantire indipendenza dalla pipeline di Genhapi e testata mediante semplici meccanismi di interfaccia web.

I risultati ottenuti dimostrano la fattibilità dell'approccio integrato, aprendo la strada a sviluppi futuri quali la gestione di ambiguità linguistiche, la continuità conversazionale in caso di interruzioni, la gestione del rumore e l'estensione a scenari multilingua.

Contents

List of Figures **5**

List of Tables **7**

General introduction **8**

1 Intelligent Conversational Interfaces **10**
1.1 State of the Art in Intelligent Conversational Interfaces 10
1.2 The Genhapi System: Architecture and Workflow 11
1.2.1 Multi-Agent Architecture 11
1.2.2 Model Context Protocol (MCP) 14
1.2.3 Operational Phases and Automation 15

2 Voice-Based Interaction Module Implementation **16**
2.1 Motivation and Objective 16
2.2 Speech Technologies 17
2.3 Module Logic 17
2.4 Genhapi Project Tree Structure 19
2.5 Frontend Implementation 21
2.6 Backend Implementation 23

3 System Test Validation **30**
3.1 Methodology 30
3.2 Results Achieved 33
3.3 Results Discussion 43

Bibliography **45**

List of Figures

1.1 Genhapi System Architecture 12

2.1 Module Integration Sequence Diagram 18
2.2 Genhapi Tree Structure 20

Listings

1.1 Genhapi JSON payload 12
1.2 MCP function insight in JSON payload 14
2.1 HTML display integration 21
2.2 JavaScript frontend vocal request transcription 22
2.3 JavaScript voice request flag in JSON payload 22
2.4 JavaScript frontend automatic autoplay integration 23
2.5 Python new audio agent module 23
2.6 Python STT service 24
2.7 Python TTS service 27
2.8 Python integration in chat view - fetching data 28
2.9 Python integration in chat view - response data 28
2.10 Python new transcribe audio script view 28
2.11 Python new delete audio played view 29

List of Tables

3.1 Test A/B tester p01 33
3.2 Test A/B tester p02 35
3.3 Test A/B tester p03 37
3.4 Test A/B tester p04 39
3.5 Test A/B tester p05 41

General introduction

Negli ultimi anni i sistemi di intelligenza artificiale basati su modelli linguistici di grandi dimensioni (Large Language Models, LLM) hanno reso possibile la realizzazione di chatbot conversazionali in grado di interpretare richieste in linguaggio naturale e supportare attività complesse di automazione. Questo anche grazie all'adozione di architetture multi-agente che hanno permesso di suddividere le responsabilità operative tra componenti specializzati, facilitando i flussi e le pipeline di automazione, decentralizzando compiti troppo grandi per singoli agenti.

All'interno di questo scenario si colloca Genhapi, un sistema conversazionale sviluppato presso il laboratorio MICC dell'Università di Firenze. Basato sul framework Agno, Genhapi è in grado di riconoscere intenti, raccogliere parametri tramite dialogo interattivo e invocare strumenti lato server (MCP tools) per automatizzare operazioni come la creazione di documenti e l'interrogazione di database. Pur offrendo un'interazione tramite chatbot piuttosto avanzata, il sistema era inizialmente limitato al solo scambio testuale.

L'obiettivo di questa tesi è stato lo sviluppo e l'integrazione di un modulo di interazione vocale per Genhapi, capace di estendere il paradigma conversazionale all'uso della voce sia in ingresso (speech-to-text) che in uscita (text-to-speech). Tale evoluzione consente di rendere l'interazione più immediata e inclusiva, e rappresenta un passo significativo verso l'adozione di sistemi conversazionali in contesti reali, dove l'interfaccia vocale può migliorare l'efficienza e le tempistiche.

Dal punto di vista tecnico il lavoro ha previsto:

- uno studio preliminare dell'architettura e della logica del framework, nonché della struttura ad albero dei file di progetto fornita dal laboratorio MICC, resa disponibile tramite un ambiente di deployment basato su Docker;
- una selezione, tra le varie alternative presentate dai relatori stessi, dei pacchetti e componenti open-source per la trascrizione del parlato (Vosk, Whisper) e la sintesi vocale (Coqui TTS);

- una breve analisi dei requisiti funzionali del modulo vocale, nonchè di una sua architettura non invasiva che permettesse di mantenere inalterato il funziamento interno di Genhapi;
- la progettazione di una semplice interfaccia lato web che agevolasse la semplicità della sperimentazione del dialogo vocale.

Chapter 1

Intelligent Conversational Interfaces

1.1 State of the Art in Intelligent Conversational Interfaces

Recentemente l'evoluzione delle tecnologie di intelligenza artificiale e dei modelli linguistici ha favorito una crescita significativa del settore delle interfacce conversazionali intelligenti. Tali sistemi si propongono di semplificare l'interazione uomo-macchina, consentendo agli utenti di accedere a funzionalità complesse attraverso il linguaggio naturale, senza dover ricorrere a comandi sintatticamente complicati o interfacce grafiche troppo specializzate.

Il mercato del *Conversational AI* è in rapida espansione: secondo stime recenti, il valore globale del settore ha raggiunto circa 11,6 miliardi di dollari nel 2024 e si prevede un incremento fino a oltre 14 miliardi nel 2025, con un tasso di crescita annuo superiore al 23% nel periodo 2025–2030 [1]. Questo trend riflette la crescente adozione delle tecnologie conversazionali da parte di imprese e organizzazioni, che vedono in tali strumenti un'opportunità per aumentare l'efficienza operativa e migliorare l'esperienza utente.

Dal punto di vista tecnologico, i moderni sistemi conversazionali si fondano su tre pilastri fondamentali:

- **riconoscimento dell'intento**, ossia la capacità di interpretare correttamente la richiesta dell'utente, confrontandola con quelle presenti nella knowledge base;
- **raccolta e validazione dei parametri**, necessaria per completare le informazioni mancanti in una richiesta incompleta;

- **invocazione sicura dei servizi backend**, permette di tradurre la richiesta in azioni concrete (ad esempio la creazione di un documento, la consultazione di un database, ecc.).

Ad oggi lo stato dell'arte delle interfacce conversazionali intelligenti prevede di adottare un'architettura multi-agente, per decentralizzare compiti complessi in task meno pesanti, da affidare a moduli specializzati (*Intent Recognizer*, *Parameter Resolver*, *API Caller*, ecc.), coordinati da un agente centrale. Altro aspetto rilevante riguarda l'integrazione con modelli linguistici di grandi dimensioni (LLM). Infatti questi modelli forniscono capacità avanzate di comprensione e generazione del linguaggio, permettendo di offrire interazioni naturali. Queste caratteristiche costituiscono le fondamenta del terreno su cui si inserisce il progetto *Genhapi*, e in generale, tutte le applicazioni mirate a migliorare l'assistenza digitale.

1.2 The Genhapi System: Architecture and Workflow

Genhapi è un sistema conversazionale multi-agente orientato all'automazione, sviluppato presso il laboratorio MICC (Media Integration Communication Center) dell'Università di Firenze. Il suo obiettivo principale è consentire agli utenti di interagire in linguaggio naturale per eseguire operazioni complesse, come la creazione di documenti e la consultazione di database, senza la necessità di conoscere comandi sintatticamente specifici.

1.2.1 Multi-Agent Architecture

Il sistema è basato sul framework Agno, che adotta un paradigma Agent/Team. Ogni agente ha un ruolo funzionale specifico (Intent Recognizer, Parameter Resolver, API Caller, Chat Memory), regole linguistiche e procedurali (prompt di servizio), e risorse di supporto (memoria conversazionale, knowledge base, ecc.). Un agente Coordinator governa il flusso, smistando i messaggi tra gli altri agenti, aggregando le risposte e garantendo coerenza semantica. Questo approccio modulare permette scalabilità, riuso e facilità di estensione a nuovi scenari.

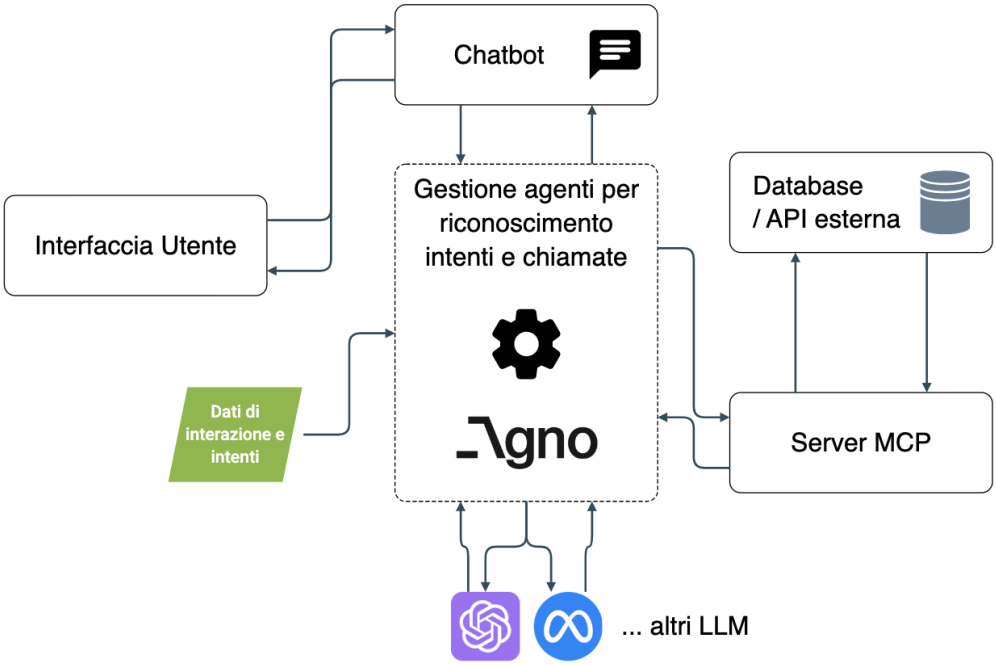


Figure 1.1: Genhapi System Architecture

In particolare Genhapi possiede una base di conoscenza cosituita da intenti formalizzati in pacchetti JSON. Ogni JSON include diversi elementi fondamentali tra cui una sezione `description`, che raccoglie formulazioni equivalenti utilizzabili dall'utente per esprimere quell'obiettivo, e un campo `required_field`, in cui vengono elencati i dati necessari per completare l'operazione. Quello che segue è un esempio di pacchetto JSON per la richiesta "*Creazione di una fattura*".

```
{
 "intents": {
   "create_invoice": {
     "description": [
       "Crea una fattura di vendita",
       "Crea una fattura per un cliente",
       "Crea una fattura del ciclo attivo"
     ],
     "required_fields": {
       "amount": {
```

```
          "type": "float",
          "description": "importo",
          "source": "user"
        },
        "date": {
          "type": "date",
          "description": "data",
          "source": "user"
        },
        "description": {
          "type": "str",
          "description": "descrizione",
          "source": "user"
        },
        "clientId": {
          "type": "int",
          "description": "cliente",
          "source": "api",
          "tool": "/customer",
          "inputType": "string",
          "outputType": "list"
        }
      },
      "target-tool": "/crete-invoice",
      "errors": {
        "001": "data fattura non valida.",
        "002": "ID del cliente sospeso."
      }
    }
  }
}
```

Listing 1.1: Genhapi JSON payload

Tutta questa struttura JSON viene completata per merito degli agenti:

- **Intent Recognizer**, che classifica la richiesta utente e popola i primi parametri;
- **Parameter Resolver**, che raccoglie e valida i parametri mancanti con un dialogo interattivo, gestendo ambiguità (es. più clienti “Rossi”) e recuperando dati da fonti esterne (es. database).

Dunque l'agente Coordinator passa l'intero JSON all'API Caller, che traduce i parametri in una chiamata sicura al server MCP.

1.2.2 Model Context Protocol (MCP)

Genhapi utilizza il Model Context Protocol (MCP) come strato di interfaccia tra il modello linguistico e i servizi gestionali. In particolare la responsabilità del client MCP è orchestrare il dialogo tra il modello e il server: riceve l'intento generato dal modello, lo mappa sul tool appropriato, e lo invoca con i parametri richiesti. Il server MCP esegue la richiesta e restituisce una risposta strutturata, che il modello può poi interpretare e tradurre in linguaggio naturale, oppure utilizzare per guidare ulteriori azioni.

Infatti per ogni intento custodito nella knowledge base, sotto la chiave `target-tool` del pacchetto JSON associato, è già indicata quale funzione MCP deve essere chiamata.

```
"create_invoice": {
    ...
    "target-tool": "/create-invoice"
}
```

Listing 1.2: MCP function insight in JSON payload

Quando quindi l'API Caller riceve l'intento JSON e i parametri validati può procedere attraverso una:

- creazione di un secondo oggetto JSON con la struttura apposita richiesta dal server MCP

$$MCP(\texttt{tool_calls}, \texttt{id}, \texttt{function.name}, \texttt{function.arguments}, ...)$$

- trasmissione sicura del payload generato sopra via HTTPS

In questo modo la logica di mappatura resta interamente dichiarativa: aggiungere un nuovo intento implica solo aggiornare il file JSON, senza modificare il codice Python dell'agente. Quindi, in sintesi, MCP fornisce una struttura chiara, estendibile e sicura per abilitare i modelli AI a operare in contesti applicativi reali, mantenendo una netta distinzione tra ciò che è linguaggio e ragionamento da una parte, e ciò che è esecuzione e integrazione tecnica dall'altra.

1.2.3 Operational Phases and Automation

Il ciclo di funzionamento di Genhapi si articola in due fasi principali:

1. **Learning**: raccolta dei log reali di interazione, estrazione degli intenti, identificazione dei parametri e mappatura delle API MCP. Questa fase alimenta la knowledge base e costituisce il "training" del sistema, senza necessità di nessun perfezionamento sui modelli;

2. **Running**: durante l'interazione, l'utente esprime una richiesta in linguaggio naturale, quindi il sistema riconosce l'intento, raccoglie i parametri, invoca il tool MCP corrispondente e restituisce la risposta in forma discorsiva.

In aggiunta, oltre alla comunicazione via API, Genhapi integra meccanismi per interagire direttamente con interfacce utente web, tramite estensioni browser che intercettano eventi (click, input, invii di form-data) e li mappano in intenti. Questa capacità è utile per sistemi legacy privi di API (si pensi ai sistemi mainframe, ossia grandi sistemi informatici che gestiscono operazioni critiche come quelle bancarie o assicurative), consentendo comunque automazione e assistenza guidata.

Nel momento di scrittura di questo lavoro lo stato dell'implementazione del progetto Genhapi è in fase di validazione in laboratorio. I test vengono condotti utilizzando ChatGPT come motore LLM e una interfaccia conversazionale di esempio, progettata per simulare scenari realistici. La validazione si concentra su alcuni casi d'uso selezionati (come la creazione di fatture o il recupero di dati anagrafici), per i quali sono stati raccolti log di interazione utente nelle fasi precedenti del progetto. Questa attività consente di verificare il corretto funzionamento della pipeline, dal riconoscimento dell'intento alla chiamata dei servizi MCP, nonché la coerenza e l'efficacia dell'esperienza conversazionale offerta dal sistema. [Dichiarazione redatta dagli sviluppatori del MICC]

Chapter 2

Voice-Based Interaction Module Implementation

2.1 Motivation and Objective

Genhapi funziona attraverso un'interfaccia testuale, un paradigma ormai consolidato: l'utente scrive un messaggio, il sistema lo elabora e fornisce una risposta. Tuttavia, l'evoluzione delle tecnologie di speech recognition e speech synthesis consente oggi di introdurre una modalità di interazione più naturale, basata sulla voce.

L'obiettivo di questo lavoro è stato quello di estendere il framework Genhapi con un Voice-Based Interaction Module capace di:

- Trasformare la voce dell'utente in testo (speech-to-text, STT), così da integrarsi nella pipeline esistente di comprensione e generazione;
- Generare risposte vocali (text-to-speech, TTS), restituendo all'utente un output non solo testuale ma anche audio.

Quindi in altre parole, il modulo vocale non sostituisce l'interfaccia testuale ma la arricchisce, consentendo all'utente di scegliere liberamente se comunicare via tastiera o tramite voce. Questa estensione rende l'interazione più immediata, soprattutto in scenari dove l'uso della tastiera è meno pratico ed efficiente (ad esempio applicazioni mobile).

Il codice integrato è stato sviluppato utilizzando l'IDE PyCharm 2025.2 e il framework backend Django. I test sono stati eseguiti in ambiente Docker Desktop su Windows 11.

2.2 Speech Technologies

L'integrazione del modulo di interazione vocale nel sistema Genhapi è iniziata con la scelta di due tecnologie complementari: **speech-to-text (STT)** per la trascrizione del parlato e **text-to-speech (TTS)** per la generazione della voce sintetica.

Gli sviluppatori del MICC hanno messo a disposizione una serie di alternative di toolkit, la quale combinazione fornisse una soluzione completamente open-source per la gestione del canale vocale. La scelta delle tecnologie utilizzate è stata guidata da una serie di considerazioni quali:

- **Compatibilità multi-lingua**, con supporto alla lingua Italiana;
- **Efficienza computazionale**, ossia la capacità di funzionare anche su macchine a risorse limitate, senza necessità di GPU;
- **Semplice integrazione con Python**, tramite API che consentono di acquisire audio da microfono e ottenere la trascrizione in tempo reale;
- **Funzionamento offline**, che garantisce maggiore privacy e indipendenza dalla connessione Internet.

A fronte di questo la scelta è ricaduta sul toolkit open-source Vosk per il riconoscimento vocale e in tempo reale (STT). I modelli Vosk infatti sono piccoli ($\sim$50 Mb), ma forniscono trascrizione continua di un ampio vocabolario, risposta a latenza zero con API di streaming, vocabolario riconfigurabile e identificazione del parlante. Possiedono inoltre collegamenti di riconoscimento vocale implementati per vari linguaggi di programmazione come Python, Java, Node.JS, C#, C++, Rust, Go e altri [2].

Coqui TTS invece è risultata la scelta più efficiente per la sintesi vocale del testo. Si tratta di un framework sviluppato con l'obbiettivo di produrre voci naturali e personalizzabili, i cui modelli possono "parlare" in 13 diverse lingue ed eseguire lo stream con una latenza $<$ 200ms. Inoltre se si è interessati alla solo sintesi vocale (senza alcun training o fine-tuning del modello), Coqui TTS risulta veramente semplice da installare in ambiente Python [3].

2.3 Module Logic

Il modulo sviluppato si deve integrare nella pipeline Genhapi in maniera non invasiva, ossia deve mantenere inalterata la logica di ragionamento e risposta del sistema. Quindi il modulo deve agire a valle pre-elaborando gli input vocali, e al termine del flusso di automazione post-elaborando gli output testuali.

L'intero processo può essere rappresentato attraverso il seguente sequence diagram:

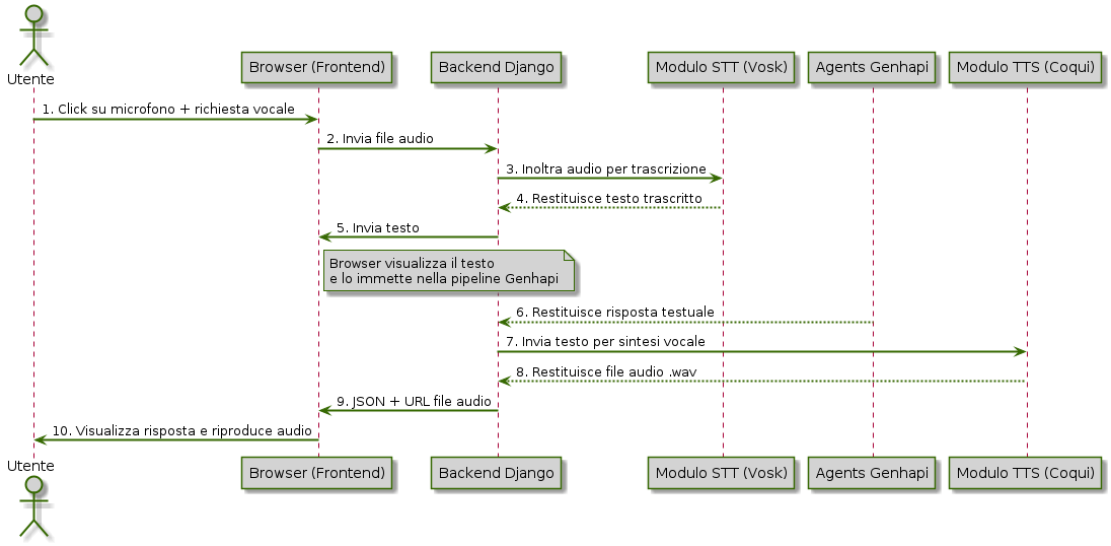


Figure 2.1: Module Integration Sequence Diagram

E può essere riassunto attraverso il diagramma di sequenza più discorsivo (dove gli attori sono marcati in grassetto):

1. L'**Utente** clicca sul bottone 🎙 e pronuncia la richiesta.
2. Il **Browser** (Frontend) invia al **Backend Django** un file audio con la registrazione della richiesta.
3. **Backend Django** inoltra l'audio al **Modulo STT** (Vosk) per la conversione audio → testo.
4. Il **Modulo STT** restituisce al **Backend Django** il testo trascritto.
5. Il **Backend Django** invia il testo al **Browser** che lo visualizza e lo immette nella pipeline **Genhapi**.
6. Gli Agents di **Genhapi** ritornano al **Backend Django** una risposta testuale.
7. Il **Backend Django** passa la risposta al **Modulo TTS** (Coqui) per la sintesi vocale.
8. Il **Modulo TTS** restituisce al **Backend Django** un file audio `.wav`.

9. Il **Backend Django** risponde al **Browser** con un pacchetto JSON + URL del file audio di risposta.

10. Il **Browser** visualizza la risposta e riproduce automaticamente l'audio per l'**Utente**.

2.4 Genhapi Project Tree Structure

In prima batutta il lavoro di tesi ha compreso un'analisi della struttura ad albero progettuale di Genhapi. Ogni file e cartella infatti è strutturata per garantire separazione logica in maniera tale da riusare e modificare il codice senza toccare la logica complessiva del framework. La comprensione di questa struttura razionale è stata di fondamentale importanza per individuare i corretti moduli già presenti da modificare/integrare e, allo stesso tempo, assicurare la "sezione" ideale della struttura in cui creare quelli nuovi. Quello che segue è una rappresentazione della *tree structure* dell'intero progetto:

```
genhapi-docker/
└── genhapi-improve-timing/
    ├── data/
    ├── web/
    │   ├── agents/
    │   ├── chatbot/
    │   ├── MCP/
    │   ├── models/
    │   ├── multi_agent_chatbot/
    │   ├── static/
    │   ├── templates/
    │   ├── tmp/
    │   ├── utils/
    │   ├── web/
    │   ├── __init__.py
    │   ├── db.sqlite3
    │   ├── Dockerfile
    │   ├── environment.yml
    │   ├── manage.py
    │   └── requirements.txt
    ├── .env
    ├── .gitignore
    └── docker-compose.yml
```



Figure 2.2: Genhapi Tree Structure

Si noti che il laboratorio MICC ha fornito il progetto non solo come insieme di file sorgenti, ma già incapsulato in un'infrastruttura containerizzata tramite Docker, così da poterlo avviare in maniera uniforme e indipendente dall'hardware o dal sistema operativo locale. Si osserva poi che la cartella `web/` costituisce il cuore del progetto. Al suo interno si trovano diversi moduli:

- **agents/**: definisce gli agenti che compongono la pipeline multi-agente di Genhapi. Qui è stato integrato il nuovo agente dedicato per i servizi STT e TTS;
- **chatbot/**: contiene le views, gli URL e la logica lato server del chabot;
- **MCP/**: modulo dedicato alla gestione Multi-Context Processing che coordina i diversi agenti;
- **models/**: directory che contiene i modelli addestrati o eventuali risorse di NLP di supporto;
- **multi_agent_chabot/**: configurazioni generali del sistema multi-agente;
- **static/**: file statici come script CSS e JavaScript. Qua risiede anche il file `chat.js` aggiornato per la gestione della registrazione e riproduzione audio;
- **templates/**: templates HMTL, come il file frontend principale `index.html`, che definiscono l'interfaccia utente;
- **utils/**: moduli di utilità condivisi, dove sono stati collocati i servizi STT e TTS.

Gli altri file che seguono sono invece indirizzati maggiormente verso le configurazioni di deployment:

- **db.sqlite3**: database SQLite di default usato in fase di sviluppo;
- **Dockerfile** e **docker-compose.yml**: definiscono l'ambiente (immagini e container) per eseguire il progetto attraverso Docker;
- **.env**: file con variabili di ambiente (come ad esempio credenziali DB, chiavi API);
- **requirements.txt** ed **environment.yml**: specificano le dipendenze Python del progetto;

- **manage.py**: entry point classico dei progetti Django, necessario per l'esecuzione del codice.

2.5 Frontend Implementation

Il lavoro svolto ha modificato l'interfaccia frontend di Genhapi, realizzata in HMTL, CSS e JavaScript veramente in maniera minimale. Infatti nella pagina principale del chatbot (`index.html`) è stato aggiunto un bottone con un'icona di un microfono, direttamente vicino al pulsante "Invia". Tale bottone microfono permette all'utente di avviare la registrazione della propria voce.

```
1 <input type="text" id="user-message" placeholder="Scrivi un messaggio...
    ">
2 <button onclick="sendMessage()">Invia</button>
3 <button id="record-audio-button" onclick="toggleRecording()"> 🎤 </button
    >
```

Listing 2.1: HTML display integration

Per acquisire l'audio direttamente dal browser è stata utilizzata la Web API `MediaRecorder`, che consente di registrare audio dal microfono resituendo un flusso dati in formato `Blob`. Per garantire una maggiore portabilità della registrazione su browser diversi viene scelto a priori il miglior formato supportato dal browser per `MediaRecorder`, attraverso una lista di MIME type (Multipurpose Internet Mail Extensions type, ossia una stringa standardizzata che descrive il formato e il contenuto di un file multimediale) come possibili candidati. Ora, durante la registrazione, i frammenti audio vengono accumolati in memoria, e al termine di questa, vengono inviati al server Django per la trascrizione. Per gestire automaticamente l'interruzione della registrazione è stato implementato un semplice algoritmo di Voice Activity Detection (VAD). L'idea è stata quella di misurare periodicamente quella che è definita come "energia" del segnale audio, per distinguere i momenti di voce da quelli di silenzio. A intervalli regolari viene calcolato il valore RMS (Root Mean Square) dei campioni audio, che fornisce una stima del "volume" del segnale. Se questo valore rimane sotto una soglia prestabilita per un intervallo di tempo consecutivo (es. 1,5 secondi), il sistema considera la presenza di silenzio prolungato e interrompe la registrazione. Per evitare interruzioni premature, i primi istanti della registrazione non vengono valutati e il contatore del silenzio viene azzerato ogni volta che viene rilevata attività vocale. In questo modo si ottiene un meccanismo semplice ma efficace per fermare automaticamente la registrazione al termine del parlato.

Dunque a questo punto l'audio grezzo viene inviato al backend tramite una richiesta POST multipart/form-data verso l'endpoint `/chatbot/audio`, che si occupa della trascrizione. Una volta ottenuta tale trascrizione testuale, il frontend la visualizza automaticamente come messaggio del chatbot e la invia automaticamente al backend tramite l'endpoint `/chat`, senza eseguire alcun clik sul bottone "Invia", riprendendo così il normale flusso di automazione di Genhapi.

```
            const formData = new FormData();
            formData.append("audio", blob, filename);

            recBtn.disabled = true;

            const genhapiRes = await fetch("/chatbot/audio", {method: "
    POST", body: formData});
            let dataRes = null;
            try {
                dataRes = await genhapiRes.json();
            } catch {
                throw new Error("Risposta STT non valida.");
            }
            if (!genhapiRes.ok || !dataRes?.success) {
                throw new Error(dataRes?.error || "Trascrizione fallita.
    ");
            }

            const text = (dataRes.text || "").trim();
            renderBotMessage("Hai detto: " + (text || "Non ho rilevato
    alcun parlato. Riprova"));
```

Listing 2.2: JavaScript frontend vocal request transcription

Per distinguere le richieste vocali da quelle testuali è stato pensato di aggiungere un flag `voice_request: true` nel payload JSON. In questo modo si rende consapevole il backend che oltre alla risposta testuale, dovrà generare anche una risposta audio.

```
fetch("/chat", {
    method: "POST",
    headers: {"Content-Type": "application/json"},
    body: JSON.stringify({
        user_message: trascrizione,
        user_token: "token123",
        voice_request: true
```

```
    })
});
```

Listing 2.3: JavaScript voice request flag in JSON payload

La risposta del backend include, oltre al testo, anche un campo `audio_url` che si riferisce a un file `.wav` generato dal modulo TTS e memorizzato in `/static/tmp/audio/`. A questo punto il frontend, dopo aver mostrato il messaggio testuale di risposta in chat, crea in maniera dinamica un elemento `<audio>` con autoplay automatico per la riproduzione vocale della risposta Genhapi.

```
if (response.audio_url) {
    const audio = new Audio(response.audio_url);
    audio.autoplay = true;
}
```

Listing 2.4: JavaScript frontend automatic autoplay integration

2.6 Backend Implementation

Il lavoro sul backend ha riguardato principalmente la creazione di moduli dedicati per il riconoscimento e sintesi vocale, nonchè l'adattamento delle viste (`views.py`) per gestire sia input che output audio. Per una minore frammentazione del codice è stato creato un modulo agente dedicato `audio_agent.py` in `web/agents/` che incapsula i due servizi:

- `STT_service.py` → servizio speech-to-text con Vosk;
- `TTS_service.py` → servizio text-to-speech con Coqui TTS.

Si osservi che in questa maniera la logica vocale resta isolata e dunque facilmente estendibile e/o migliorabile.

```
# web/agents/audio_agent.py
from utils.STT_service import transcribe_audio_to_text
from utils.TTS_service import generate_audio

class AudioAgent:
    """
    Agente specializzato per i servizi speech -> text (STT)
       e text -> speech (TTS)
```

```
8       """
9
10      def transcribe_file(self, file_path: str) -> str:
11          """
12          Utilizza un modello VOSK per trascrivere la richista
                vocale dell'utente nel chatbot.
13          """
14          return transcribe_audio_to_text(file_path)
15
16      def synthesize(self, text: str) -> str:
17          """
18          Ritorna percorso assoluto del file audio .wav
               associato alla risposta Genhapi.
19          """
20          return generate_audio(text)
21
22  audio_agent = AudioAgent()
```

Listing 2.5: Python new audio agent module

Nello specifico il servizio STT, implementato con Vosk, permette una trascrizione multilingua. Il modulo `STT_service.py` associato si occupa di:

1. Scaricare il modello Vosk e memorizzarlo nella cartella `web/models`, dunque effettuare la sua inizializzazione;

2. Convertire i file ricevuti dal browser (in diversi formati tra cui `.webm`, `.ogg`, `.wav`, ecc.) in file audio WAV mono 16kHz, attraverso la libreria di manipolazione audio `pydub`, direttamente scaricabile in ambienti Python. Questo step garantisce la conversione in un file audio con una maggiore compatibilità con i modelli Vosk;

3. Effettuare la trascrizione utilizzando il riconoscitore `KaldiRecognizer`, ossia il motore di speech recognition incapsulato da Vosk che contiene tutta l'infrastruttura di decodifica acustica (reti neurali, decodifica statistica). Il risultato di questo step è una stringa testo pulita associata alla richiesta utente che viene inserita nella normale pipeline Genhapi.

```
1
2   #-------------------------------------------------------------
3   # Scelta del modello VOSK e creazione delle cartelle
        necessarie
```

```
4  BASE_DIR = Path(__file__).resolve().parents[1]
5  MODELS_DIR = BASE_DIR / "models"
6  VOSK_MODEL_NAME = "vosk-model-small-it-0.22"
7  VOSK_MODEL_PATH = MODELS_DIR / VOSK_MODEL_NAME
8  VOSK_URL = "https://alphacephei.com/vosk/models/vosk-model-
       small-it-0.22.zip"
9  #------------------------------------------------------------
10
11 #------------------------------------------------------------
12 # Download e inizializzazione del modello scelto
13 def manage_vosk_model():
14
15     with urllib.request.urlopen(VOSK_URL) as r, open(tmp_zip
           , "wb") as f:
16         while True:
17             chunk = r.read(1024 * 1024)
18             if not chunk:
19                 break
20             f.write(chunk)
21
22     tmp_zip.unlink(missing_ok=True)
23
24 manage_vosk_model()
25 vosk_model = Model(str(VOSK_MODEL_PATH))
26
27 # Inizializzazione del riconoscitore Kaldi
28 rec = KaldiRecognizer(vosk_model, 16000)
29 #------------------------------------------------------------
30
31 def transcribe_audio_to_text(audio_file_path: str) -> str:
32
33     audio = AudioSegment.from_file(audio_file_path)
34     audio = audio.set_channels(1).set_frame_rate(16000).
           set_sample_width(2)
35
36     tmp_wav = Path(audio_file_path).with_suffix(".mono16k.
           wav")
37     audio.export(tmp_wav, format="wav")
38
39     wf_mono = wave.open(str(tmp_wav), "rb")
40
41     # Lista per accumulare i risultati della trascrizione
42     transcribe_text_list = []
```

```
43
44      while True:
45          data = wf_mono.readframes(4000)
46          if len(data) == 0:
47              break
48          if rec.AcceptWaveform(data):
49              result = json.loads(rec.Result())
50              transcribe_text_list.append(result["text"])
51
52      # Gestione dell'ultimo frammento di audio
53      final_result = json.loads(rec.FinalResult())
54      transcribe_text_list.append(final_result["text"])
55
56      # Unione dei risultati in un'unica stringa
57      complete_text = ' '.join(transcribe_text_list)
58
59      return complete_text.strip()
```

Listing 2.6: Python STT service

Per quanto riguarda invece il modulo `TTS_service.py`, questo si occupa di inizializzare uno tra i modelli single-speaker (in italiano) presenti dentro il pacchetto `TTS`. Dunque utilizza tale modello per trasformare una stringa di testo ricevuta in input in un file audio `.wav` che memorizza in `/static/tmp/audio/` con denominazione `GenhapiReply_Date(YYYYMMDD)_Time(HHMMSS).wav`. Questa struttura logica garantisce che ogni risposta della pipeline Genhapi abbia un file audio distinto, servibile tramite Django, e quindi permette una maggiore qualità del debug e testing del servizio, almeno nelle prime fasi di implementazione del codice. In fase di test è sorta una problematica legata alla poca precisione in lingua italiana dei modelli `glow-tts` utilizzati (leggeri e portabili, ma evidentemente non precisissimi). Infatti quando nel testo appaiono numeri, date, orari il modello non riesce a pronunciarli correttamente durante il parlato. Per cercare di rimediare a questo problema e fornire risposte più pulite e comprensibili possibile, è stato pensato di effettuare a valle (ossia prima di somministrare al modello tts la stringa di testo) una normalizzazione del testo in lingua italiana. E' stato così necessario appellarsi all'uso delle espressioni regolari per intercettare nelle scritture numeri isolati, date, orari e codici per convertirli in lettere attraverso la libreria `num2words`, facilmente scaricabile in ambienti Python. Ad esempio:

- **orari**: 14:05 → "quattordici e cinque";
- **date**: 23/09/2025 → "ventitre settembre duemilaventicinque";

- **numeri isolati**: 1200 euro → “milleduecento euro”;
- **codici alfanumerici**: IT12345678 → “i ti uno due tre quanttro cinque sei sette otto”;
- **codici numerici lunghi**: 76997142645 → “sette sei nove nove sette uno quattro due sei quattro cinque”;
- **parole con separatori**: anno-mese-giorno → “anno mese giorno”.

```
BASE_DIR = Path(__file__).resolve().parents[1]
OUT_DIR = BASE_DIR / "static" / "tmp" / "audio"
OUT_DIR.mkdir(parents=True, exist_ok=True)

MODEL_NAME = os.environ.get("TTS_MODEL_NAME", "").strip() or
    "tts_models/it/mai_male/glow-tts"

tts = TTS(model_name=MODEL_NAME, progress_bar=False, gpu=
   False)

def generate_audio(text: str) -> str:
    """
    Ritorna il percorso assoluto del file WAV generato come
       risposta alla richiesta utente vocale.
    """
    text_norm = normalize_italian(text)

    now = datetime.now()
    file_wav_name = f"GenhapiReply_Date({now.strftime('%Y%m%
       d')})_Time({now.strftime('%H%M%S')}).wav"
    out_path = OUT_DIR / file_wav_name

    tts.tts_to_file(text=text_norm, file_path=str(out_path))
    return str(out_path)
```

Listing 2.7: Python TTS service

A questo punto è stato necessario andare a modificare la vista `chat` di `views.py` in modo tale da integrare quanto fatto sopra. Nel dettaglio la vista è stata estesa per:

- verificare se la richiesta utente è vocale, attraverso il flag `voice_request`;
- generare non solo la risposta testuale, ma anche un file audio corrispondente attraverso la chiamata `audio_agent.synthesize()`.

```python
response_audio_url = None
if voice_request and content:
    response_wav = audio_agent.synthesize(str(content))
    response_wav_name = os.path.basename(response_wav)
    response_audio_url = f"/static/tmp/audio/{
        response_wav_name}"
```

Listing 2.8: Python integration in chat view - fetching data

E dunque la risposta JSON include sia il testo della risposta Genhapi che l'URL del file `.wav` associato:

```python
return JsonResponse({
    "status": "complete",
    "response": str(content) if content else "Operazione
        completata.",
    "audio_url": response_audio_url
})
```

Listing 2.9: Python integration in chat view - response data

E' stato poi necessaria la scrittura di una nuova vista, `transcribe_audio_view`, per gestire l'endpoint `/chatbot/audio`, ossia per occuparsi dell'upload dei file vocali direttamente dal frontend e gestire la loro trascrizione. Questa infatti riceve un file audio, lo converte in `WAV` e lo trascrive tramite `audio_agent.transcribe_file()`, restituendo il testo come risposta JSON.

```python
@csrf_exempt
def transcribe_audio_view(request):
    if request.method != 'POST' or 'audio' not in request.
        FILES:
         audio_file = request.FILES['audio']
         ...
         transcript = audio_agent.transcribe(tmp_path)
         return JsonResponse({"success": False, "error": str(
            e)}, status=500)
```

Listing 2.10: Python new transcribe audio script view

Infine è stata implementata anche una vista `delete_audio_played`, chiamata lato frontend ogniqualvolta la riproduzione del file audio di riposta `GenhapiReply_Date(YYYYMMDD)_Time(HHMMSS).wav` termina, necessaria per eliminare la risorsa .wav in modo tale da prevenire inutili accumoli nella cartella `static/tmp/audio`.

```python

@csrf_exempt
def delete_audio_played(request):
    """
    Cancella in modo sicuro il file wav una volta che il frontend ne conferma la riproduzione.
    """

    ...

    # ricostruisco il path assoluto
    file_path = os.path.abspath(os.path.join(AUDIO_DIR, filename))
    try:
        if os.path.commonpath([AUDIO_DIR, file_path]) != AUDIO_DIR:
            return JsonResponse({"success": False, "error": "Path non consentito."}, status=400)
    except Exception:
        return JsonResponse({"success": False, "error": "Validazione path fallita."}, status=400)

    # cancellazione del file audio
    if os.path.exists(file_path):
        try:
            os.remove(file_path)
            return JsonResponse({"success": True, "deleted": filename})
        except Exception as e:
            return JsonResponse({"success": False, "error": f"Delete failed: {e}"}, status=500)
    else:
        # se gia' cancellato o mai esistito -> okay
        return JsonResponse({"success": True, "deleted": None, "note": "File non trovato"})
```

Listing 2.11: Python new delete audio played view

Chapter 3

System Test Validation

3.1 Methodology

Per valutare il funzionamento e testare l'efficacia dell'integrazione vocale sviluppata per il sistema Genhapi, è stato adottato un protocollo sperimentale di tipo A/B comparativo basato su task specifici. L'obbiettivo era quello di misurare le differenze di prestazioni e usabilità tra:

- **Condizione A (baseline)**: interazione testuale o tradizionale con il sistema Genhapi, quindi attraverso digitazione di input e ricezioni di riposte nel solo formato testuale;
- **Condizione B (sperimentale)**: interazione vocale, ossia attraverso input vocale con riconoscimento automatico della voce (STT) e output parlato tramite sintesi TTS.

  I test vocali sono stati effettuati in ambienti silenziosi, o quantomeno quasi privi di rumori di sottofondo.

Causa la natura prototipale del sistema, la validazione è stata condotta da un unico tester (l'autore stesso). Nello specifico sono stati definiti sei diversi task che coprono diverse tipologie di interazione con il sistema, come creazione di documenti, interrogazioni al database, operazioni di modifica su record, esplorazione delle funzionalità disponibili e così via. Per simulare un testing qualitativo vengono creati 5 utenti (p01, p02, p03, p04, p05) virtuali attraverso la generazione di cinque diverse formulazioni per lo stesso intento a cui ogni task è associato. Ad esempio il partecipante tester **p01** testa il sistema attraverso i task:

1. **T1**: "Crea una fattura per il cliente Mario Rossi in data odierna di 330 euro con descrizione balle di fieno";

2. **T2**: “Cerca tutte le fatture in uscita per il cliente Mario Rossi dal 20/01/2024 ad oggi”;
3. **T3**: “Forniscimi tutte le informazioni anagrafiche e dettagliate relative al cliente Mario Rossi”;
4. **T4**: “Modifica l’indirizzo di fatturazione del cliente Mario Rossi a via della quiete 92, Roma”;
5. **T5**: “Cambia lo stato della fattura con ID 10004 per cliente Mario Rossi a pagata”;
6. **T6**: “Visualizza tutte le fatture in uscita per il cliente Mario Rossi dal 30/05/2024 ad oggi con importo superiore a mille euro”.

Il partecipante **p02**:

1. **T1**: “Genera una nuova fattura per il cliente Maria Rossi, in data odierna, per un importo di 450 euro con causale: trasporto merci”;
2. **T2**: “Mostrami tutte le fatture emesse per il cliente Maria Rossi dal 01/02/2020 fino a oggi”;
3. **T3**: “Recupera tutti i dati anagrafici relativi al cliente Maria Rossi”;
4. **T4**: “Aggiorna l’indirizzo di fatturazione del cliente Maria Rossi a via Verdi 15, Milano”;
5. **T5**: “Segna come saldata la fattura con ID 10000 per il cliente Maria Rossi”;
6. **T6**: “Visualizza le fatture emesse per il cliente Maria Rossi dal 01/02/2020 a oggi con importo superiore a 1.500 euro”.

Il partecipante **p03**:

1. **T1**: “Emetti una fattura intestata al cliente Gianni Russi, datata oggi, per un valore di 275 euro con descrizione: consulenza tecnica”;
2. **T2**: “Ricercare tutte le fatture in uscita per il cliente Gianni Russi a partire dal 15/03/2019 fino ad oggi”;
3. **T3**: “Mostrami tutte le informazioni anagrafiche del cliente Gianni Russi”;

4. **T4**: “Modifica l’indirizzo di fatturazione del cliente Gianni Russi in corso Garibaldi 88, Torino”;
5. **T5**: “Aggiorna lo stato a pagata per la fattura con ID 10002 del cliente Gianni Russi”;
6. **T6**: “Mostra l’elenco delle fatture in uscita per Gianni Russi dal 01/07/2020 ad oggi con importo oltre 1.100 euro”.

Il partecipante **p04**:

1. **T1**: “Crea una fattura intestata al cliente Giacomo Russola con data di oggi, importo 600 euro e descrizione: servizio di pulizia”;
2. **T2**: “Trova tutte le fatture emesse al cliente Giacomo Russola dal 05/04/2020 fino ad oggi”;
3. **T3**: “Recupera e mostra tutti i dati anagrafici relativi a Giacomo Russola”;
4. **T4**: “Aggiorna l’indirizzo di fatturazione del cliente Giacomo Russola a via Mazzini 120, Bologna”;
5. **T5**: “Imposta come pagata la fattura con ID 10002 relativa a Giacomo Russola”;
6. **T6**: “Visualizza tutte le fatture emesse per Giacomo Russola dal 20/06/2020 ad oggi con importo maggiore di 900 euro”.

Il partecipante **p05**:

1. **T1**: “Emetti una nuova fattura per il cliente Giacomo Resso, datata oggi, pari a 1.200 euro con causale: fornitura materiali edili”;
2. **T2**: “Elenca tutte le fatture emesse nei confronti di Giacomo Resso dal 12/02/2022 fino a oggi”;
3. **T3**: “Mostrami i dati anagrafici e le informazioni dettagliate relative a Giacomo Resso”;
4. **T4**: “Modifica l’indirizzo di fatturazione del cliente Giacomo Resso a via Dante 45, Firenze”;
5. **T5**: “Aggiorna lo stato a saldato per la fattura con ID 10045 del cliente Giacomo Resso”;

6. **T6**: “Mostrami l’elenco delle fatture emesse per Giacomo Resso dal 01/05/2021 ad oggi con importo superiore a 3.000 euro”.

Ogni task viene somministrato al sistema singolarmente, dunque vengono raccolte alcune metriche validanti quali:

- *Intento*: riconoscimento o meno dell’intento del task;
- *Tempo*: tempo totale per completare il task, misurato in secondi con un cronometro (dall’inizio dell’immissione dell’input alla generazione della risposta);
- *Errori Input*: presenza di errori di input, di trascrizione o necessità di riformulazione;
- *Note qualitative*: osservazioni su eventuali difficoltà o limiti funzionali.

Ogni task è stato poi svolto sia sotto la condizione A che in condizione B, così da permettere un confronto diretto (within-subjects).

3.2 Results Achieved

Table 3.1: Test A/B tester p01

Task	Condizione	Intento	Tempo (sec)	Errori Input	Note
T1: Creazione fattura	A	Sì	72.84	No	–
T1: Creazione fattura	B	Sì	46.65	No	–
T2: Ricerca fatture cliente	A	Sì	83.80	No	–

Continua nella pagina successiva

Task	Condizione	Intento	Tempo (sec)	Errori Input	Note
T2: Ricerca fatture cliente	B	Sì	59.38	No	–
T3: Info anagrafiche cliente	A	Sì	37.92	No	–
T3: Info anagrafiche cliente	B	No	28.02	Sì	Il sistema STT ha avuto difficoltà a comprendere e dunque trascrivere correttamente la parola "*Forniscimi*". Al secondo tentativo, pronunciando "*Scrivimi*", l'intento è stato riconosciuto.
T4: Modifica indirizzo	A	Sì	46.99	No	Il sistema ha riconosciuto l'intento, ma gli strumenti di aggiornamento indirizzo non sono ancora disponibili nel backend.
T4: Modifica indirizzo	B	Sì	35.86	No	Come sopra: riconoscimento intento corretto, funzionalità di aggiornamento non ancora implementata.
T5: Cambio stato fattura	A	Sì	27.33	No	Il sistema ha riconosciuto l'intento; la modifica dello stato delle fatture non è ancora disponibile.

Continua nella pagina successiva

Task	Condizione	Intento	Tempo (sec)	Errori Input	Note
T5: Cambio stato fattura	B	No	30.21	Sì	Inizialmente l'intento non è stato riconosciuto correttamente. Comprende l'intento quando si scandisce l'ID cifra per cifra (ID 10004 = uno zero zero zero quattro).
T6: Ricerca con vincoli multipli	A	Sì	90.94	No	L'intento è stato correttamente riconosciuto anche se il sistema ha richiesto conferma di alcuni parametri (es. client ID, periodo, importo minimo) prima di procedere.
T6: Ricerca con vincoli multipli	B	Sì	82.62	No	Come sopra: richiesta di conferma parametri aggiuntivi prima dell'esecuzione.

Table 3.2: Test A/B tester p02

Task	Condizione	Intento	Tempo (sec)	Errori Input	Note
T1: Creazione fattura	A	Sì	76.88	No	–
T1: Creazione fattura	B	Sì	44.55	No	–

Continua nella pagina successiva

Task	Condizione	Intento	Tempo (sec)	Errori Input	Note
T2: Ricerca fatture cliente	A	Sì	85.42	No	–
T2: Ricerca fatture cliente	B	Sì	80.17	No	–
T3: Info anagrafiche cliente	A	Sì	29.54	No	–
T3: Info anagrafiche cliente	B	No	36.23	No	–
T4: Modifica indirizzo	A	Sì	54.37	No	Riconosce intento e parametri, chiede ulteriore conferma per procedere.
T4: Modifica indirizzo	B	Sì	26.55	No	–
T5: Cambio stato fattura	A	Sì	37.53	No	Riconosce solo l'intento, funzioni non ancora implementate in sviluppo.
T5: Cambio stato fattura	B	Si	33.78	No	Riconosce intento, chiede una data di aggiornamento.

Continua nella pagina successiva

Task	Condizione	Intento	Tempo (sec)	Errori Input	Note
T6: Ricerca con vincoli multipli	A	Sì	76.74	No	Riconosce intento, non riceve dalle APIs dettagli sulle fatture trovate.
T6: Ricerca con vincoli multipli	B	Sì	40.50	No	Riconosce intento, chiede conferma dei dati prima di procedere.

Table 3.3: Test A/B tester p03

Task	Condizione	Intento	Tempo (sec)	Errori Input	Note
T1: Creazione fattura	A	Sì	62.55	No	–
T1: Creazione fattura	B	Sì	60.48	Si	Errore di trascrizione della parola "*Emetti*", tuttavia l'intento viene riconosciuto uguale.
T2: Ricerca fatture cliente	A	Sì	54.68	No	–
T2: Ricerca fatture cliente	B	Sì	53.27	No	–

Continua nella pagina successiva

Task	Condizione	Intento	Tempo (sec)	Errori Input	Note
T3: Info anagrafiche cliente	A	Sì	39.48	No	–
T3: Info anagrafiche cliente	B	Si	38.19	No	–
T4: Modifica indirizzo	A	Sì	53.46	No	Intento riconosciuto, il sistema al momento non dispone dell'operazione richiesta.
T4: Modifica indirizzo	B	Sì	37.26	No	Intento riconosciuto, il sistema al momento non dispone dell'operazione richiesta.
T5: Cambio stato fattura	A	Sì	50.26	No	Intento riconosciuto, il sistema al momento non dispone dell'operazione richiesta.
T5: Cambio stato fattura	B	Si	44.68	No	–
T6: Ricerca con vincoli multipli	A	Sì	81.40	No	–

Continua nella pagina successiva

Task	Condizione	Intento	Tempo (sec)	Errori Input	Note
T6: Ricerca con vincoli multipli	B	Sì	85.73	No	–

Table 3.4: Test A/B tester p04

Task	Condizione	Intento	Tempo (sec)	Errori Input	Note
T1: Creazione fattura	A	Sì	74.47	No	–
T1: Creazione fattura	B	Sì	52.35	No	–
T2: Ricerca fatture cliente	A	Sì	131.62	No	–
T2: Ricerca fatture cliente	B	Sì	82.73	No	–
T3: Info anagrafiche cliente	A	Sì	35.29	No	–
T3: Info anagrafiche cliente	B	Si	35.09	No	Riconosce l'intento, chiede conferma del nome del cliente prima di procedere.

Continua nella pagina successiva

Task	Condizione	Intento	Tempo (sec)	Errori Input	Note
T4: Modifica indirizzo	A	Sì	60.12	No	Riconosce l'intento, chiede conferma dei dati prima di procedere.
T4: Modifica indirizzo	B	Sì	39.98	No	Riconosce l'intento, risponde che attualmente l'operazione di aggirnamento record non è disponibile.
T5: Cambio stato fattura	A	Sì	72.33	No	Riconosce l'intento anche se non riesce a recuperare i dati della fattura.
T5: Cambio stato fattura	B	Si	67.34	No	Riconosce l'intento, tenta di eseguire la chiamata per completare l'operazione.
T6: Ricerca con vincoli multipli	A	Sì	75.47	No	–
T6: Ricerca con vincoli multipli	B	Sì	53.60	No	–

Table 3.5: Test A/B tester p05

Task	Condizione	Intento	Tempo (sec)	Errori Input	Note
T1: Creazione fattura	A	Sì	73.26	No	–
T1: Creazione fattura	B	Sì	157.73	Si	Non riesce a trascrivere correttamente il nome “Giacomo Resso” e lo confonde con "Giacomo Resto" o “Giacomo Rosso”. Tuttavia riconosce l’intento, quindi visualizza una serie di nomi simili per una conferma prima di procedere.
T2: Ricerca fatture cliente	A	Sì	75.24	No	Riconosce l’intento, non riesce a recuperare informazioni sulle fatture disponibili.
T2: Ricerca fatture cliente	B	Sì	57.26	No	Nella richiesta si identifica il cliente con il proprio ID invece del nome.
T3: Info anagrafiche cliente	A	Sì	52.45	No	–
T3: Info anagrafiche cliente	B	Si	36.45	No	Nella richiesta si identifica il cliente con il proprio ID invece del nome.

Continua nella pagina successiva

Task	Condizione	Intento	Tempo (sec)	Errori Input	Note
T4: Modifica indirizzo	A	Sì	39.48	No	Riconosce l'intento, suggerisce di contattare l'amministratore in quanto ad ora il sitema non dispone del tool per l'aggiornamento di record.
T4: Modifica indirizzo	B	Sì	52.9	No	Nella richiesta si identifica il cliente con il propri ID invece del nome. Riconosce l'intento ed esegue un recap dei dati prima di procedere.
T5: Cambio stato fattura	A	Sì	76.67	No	Riconosce l'intento, tenta di eseguire e riscontra un problema tecnico. Suggerisce di contattare il supporto tecnico.
T5: Cambio stato fattura	B	Si	67.98	No	Nella richiesta si identifica il cliente con il proprio ID invece del nome. Riconosce l'intento ed esegue un recap dei dati prima di procedere.
T6: Ricerca con vincoli multipli	A	Sì	58.25	No	Riconosce l'intento, non riesce a recuperare informazioni sulle fatture disponibili.

Continua nella pagina successiva

Task	Condizione	Intento	Tempo (sec)	Errori Input	Note
T6: Ricerca con vincoli multipli	B	Sì	53.27	No	Nella richiesta si identifica il cliente con il proprio ID invece del nome. Riconosce l'intento ed esegue un recap dei dati prima di procedere.

3.3 Results Discussion

I risultati ottenuti e stilati nelle tabelle precedenti mostrano visivamente alcune tendenze. Infatti, anche se non sono stati effettuati test con utenti finali, quindi i dati ottenuti hanno una valenza dimostrativa più che statistica, è possibile intravedere alcuni pattern nei valori raggiunti. Ad esempio si osserva il riconoscimento dell'intento con successo di ogni task in entrambe le condizioni. Questo dunque conferma che il modulo vocale integrato non compromette le capacità funzionali del sistema, me ne preserva l'affidabilità. Inoltre i tempi medi di riconoscimento risultano inferiori nella condizione vocale rispetto a quella testuale, con riduzioni variabili (eccetto qualche caso eccezionale come per i task T5 in tabella 3.1, T3 in tabella 3.2, T6 in tabella 3.3 e T1, T4 in tabella 3.5) tra il $\sim$10% e il $\sim$35% a seconda del task. Si osservi ad esempio che in tabella 3.1 per il task T1 (creazione fatture) si passa da 72.84s in condizione A a 46.65s sotto la condizione B, oppure per il task T2 (ricerca fatture) da 83.80s a 59.38s. Questo suggerisce che, come era prevedibile, l'interazione vocale può snellire i tempi di input, rendendo più fluida l'esperienza utente. Altro parametro degno di un'analisi più approfondita è la colonna degli **Errori Input**. Infatti questa indica che in condizione testuale non sono stati registrati errori, mentre si sono verificati 2 errori di trascrizione STT (sempre concentrandoci sulla tabella 3.1) per i task T3 e T5. Anche se facilmente trattabili entrambi con riformulazione o maggiore chiarezza dell'input, questo mette in luce un limite attuale del modulo STT (Vosk), che appunto può risentire di pronunce particolari o termini meno comuni nella lingua italiana, nonchè di ambienti non troppo silenziosi.
I feedback ottenuti attraverso la validazione del modulo potrebbero guidare verso ulteriori ottimizzazioni, quando questo è utilizzato in scenari realistici, come ad esempio:

- **Robustezza in ambienti reali**: gestione del rumore e voci sovrapposte;

- **Miglioramento del servizio STT per la lingua italiana**: maggiore precisione sulla pronuncia di date (in formati diversi), codici alfanumerici e non, orari e numeri puri grandi;
- **Continuità conversazionale**: gestione delle interruzioni durante la registrazione vocale e di dialoghi più lunghi;
- **Supporto multilingua**: garanzia di estensione a scenari internazionali.

In ogni modo questa serie di risultati preliminari, seppur ottenuti con un singolo partecipante che simula 5 tester virtuali, dimostrano il potenziale del modulo vocale circa l'incremento dell'usabilità e dell'efficienza del sistema Genhapi, ponendo le basi per future validazioni con veri utenti finali e su campioni più ampi.

Bibliography

[1] Grand View Research, "Conversational ai market size, share & trends analysis report by component (solutions, managed services, professional services), by type, by deployment, by technology, by end user, by region, and segment forecasts, 2025–2030," 2025, market Research Report.

[2] alphacep. (2025) vosk-api. [Online]. Available: https://github.com/alphacep/vosk-api

[3] C. AI. (2025) Coqui tts documentation. [Online]. Available: https://docs.coqui.ai/en/latest/